

How the data sits in SQL, and why

The companion to *How the data DOCKET collects will be used*. That document said what is collected and what it is for. This one shows the actual tables, column by column, and explains the reason behind each shape, so a reader can judge whether the database can carry the uses promised.

Database PostgreSQL on Render (SQLite for local development)

Tables 27 in the application schema, plus Django's user table

Source Generated from the DOCKET models on 14 September 2026

CONTENTS

1. Seven conventions every table follows
2. How the tables relate
3. The vendor register
4. Tenders, rounds and clarifications
5. Bids, auction bids and documents
6. The audit trail
7. The finance mirror
8. People, roles and access
9. System tables
10. The questions, as queries
11. What is deliberately not a column

1. Seven conventions every table follows

Read these first. Almost every "why is that column shaped like that" below comes back to one of them.

Text primary keys, generated by the app

Every business row has `id varchar(16)`: a one-letter prefix and nine random characters, such as `t4k9x2mq1z` for a tender or `e7ab...` for an audit event.

Why. A counter leaks how many tenders you have run and lets anyone guess the next one. Random ids are safe in URLs and emails, and they can be minted inside the same transaction that writes the audit event, with no round trip to ask the database for a number.

Time is `bigint` epoch milliseconds

`published_at`, `deadline`, `paid_at`, `at`: all the same type, all UTC milliseconds.

Why. It is exactly what the browser's clock produces, so the API sends numbers and nothing is converted or mis-zoned on the way. "Days to pay" is a subtraction. The cost is readability in a raw query, which one `to_timestamp(at/1000)` fixes.

Money is `bigint` whole naira

`budget`, `amount`, `awarded_amount`, `unit_cost`: integers, in ₦.

Why. Floating point cannot add money without rounding drift, and a savings figure that is off by kobo after a sum is a savings figure somebody will query. Integers add exactly. At contract scale kobo carry no information, so the smallest unit is the naira.

`NULL` means "not yet" or "not recorded"

`opened_at` is `NULL` until the opening.

`baseline` is `NULL` when nobody recorded one. `bid.amount` is `NULL` while sealed.

Why. Zero is a number and would be summed, compared and reported as a fact. A baseline of ₦0 says "this used to be free"; `NULL` says "we do not know", and the savings logic falls back to budget and says so. Text columns instead default to `''`, because empty text has no such ambiguity.

`jsonb` for shapes the screen owns

`criteria`, `lines`, `invited`, `scores`, `deadline_changes`, `change_orders` are JSON columns, not child tables.

Why. Each is read whole on every render and written whole once, and none is queried on its own. Ten join tables would add ten joins to every page for lists that are never filtered independently. Postgres can still index and query inside them when a report needs to.

Names in audit rows, keys everywhere else

`event.actor`, `round.created_by`, `invoice.approved_by` are names as text. `tender.owner_id`, `contract.supplier_id` are foreign keys.

Why. An audit record must say who did it in the words that were true then; if the person is renamed or deleted, the record must not change. A workload report must add things up by person, and names are not keys. Two questions, two column types.

Deletion never erases history

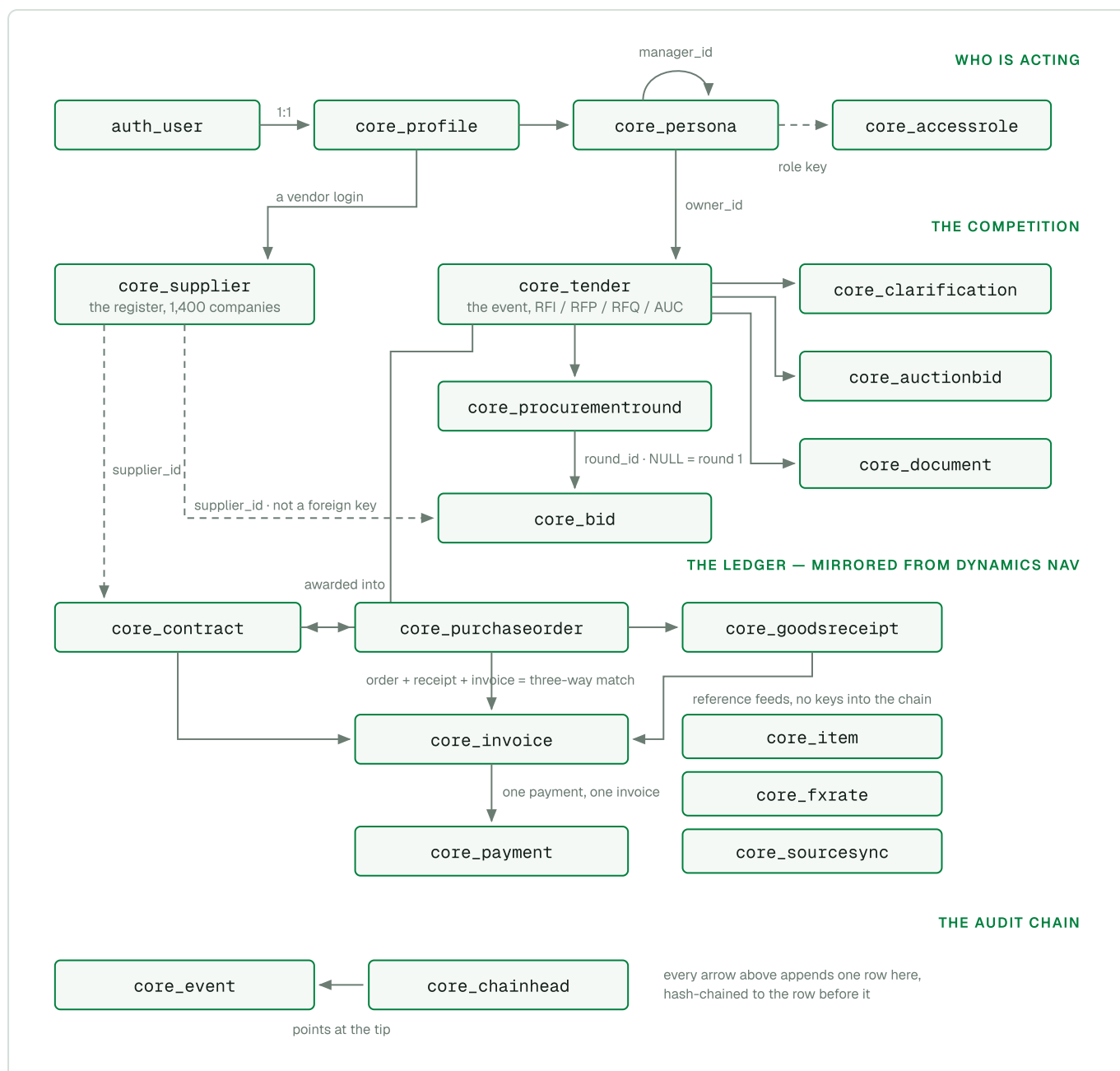
People are `ON DELETE SET NULL`. Bids cascade only under their tender.

`bid.supplier_id` and `event.tender_id` are plain text, not foreign keys.

Why. A tender outlives the buyer who ran it. A bid is a fact about what a vendor offered, and re-importing the register must never be able to cascade-delete it, so the link is deliberately not enforced by the database; the import checks it instead and refuses to delete a referenced vendor.

2. How the tables relate

Solid lines are foreign keys the database enforces. The two dashed links are text references the application maintains on purpose, for the reason in the last convention above.



Dashed connectors are text references the application maintains rather than keys the database enforces. Token, notification and failed-login tables hang off **auth_user** and are left off the map to keep it readable.

3. The vendor register

core_supplier

```
CREATE TABLE core_supplier (
  id          varchar(16) PRIMARY KEY,    -- slug of the company name
  name        varchar(120) NOT NULL,
  code        varchar(24) NOT NULL DEFAULT '', -- NAV / RP vendor code
  category    varchar(60) NOT NULL,       -- normalised bucket the app filters on
  subcategory varchar(60) NOT NULL DEFAULT '', -- '' = rules could not place it
  classification varchar(140) NOT NULL DEFAULT '', -- the register's own words
  location    varchar(60) NOT NULL,
  -- contact
  contact_person varchar(140) NOT NULL DEFAULT '',
  contact_email  varchar(200) NOT NULL DEFAULT '',
  phone          varchar(120) NOT NULL DEFAULT '',
  address        varchar(300) NOT NULL DEFAULT '',
  payment_terms  varchar(80) NOT NULL DEFAULT '',
  -- onboarding: pending -> invited -> registered
  source        varchar(12) NOT NULL,      -- self | invite | buyer | import
  invited_at    bigint,
  invite_count  integer NOT NULL DEFAULT 0,
  invite_error  varchar(200) NOT NULL DEFAULT '',
  registered_at bigint,
  -- verification: unverified -> verified | rejected | suspended
  prequalified  boolean NOT NULL DEFAULT false,
  verified_at   bigint,
  verified_by   varchar(120) NOT NULL DEFAULT '',
  rejected_reason varchar(300) NOT NULL DEFAULT '',
  suspended     boolean NOT NULL DEFAULT false,
  suspended_reason varchar(300) NOT NULL DEFAULT '',
  suspended_at  bigint,
  -- earned here, never overwritten by an import
  rating        double precision NOT NULL DEFAULT 0,
  perf          jsonb NOT NULL DEFAULT '{}', -- {onTime, quality}
  docs          jsonb NOT NULL DEFAULT '[]', -- [{name, expiry}]
  exposure_limit bigint NOT NULL DEFAULT 0,  -- 0 = no limit on file
  registry      jsonb NOT NULL DEFAULT '{}', -- TIN, bank name, *****0225, remarks, original cells
);
```

WHY IT IS SHAPED THIS WAY

The id is a slug of the name, so the same company from the spreadsheet lands on the same row every import. That is what makes the import a replace rather than an append: run it twice and you have one of everything.

Two verification facts, not one flag. A vendor can be fully registered and entirely unverified. Registration is three timestamps; verification is a boolean plus when, by whom, and two reasons. A single status column could not say "verified, but suspended for a live dispute, and must come back verified when it lifts".

Category and classification both survive. The register's 612 distinct wordings are collapsed to 23 buckets by rules. The bucket is what the screen filters on; the original wording is kept beside it because a mapping nobody can audit is a mapping nobody should trust.

The bank account is never a column. Only the last four digits, inside `registry`. DOCKET awards tenders and does not pay invoices, so 1,400 account numbers have no reason to be in this database or reach a browser.

Exposure limit 0 means "none set". The finance view reports it as "no limit on file", never as a limit of nothing. Confusing the two tells Finance every vendor is in breach.

4. Tenders, rounds and clarifications

core_tender

```
CREATE TABLE core_tender (
  id          varchar(16) PRIMARY KEY,
  ref         varchar(40) NOT NULL UNIQUE,      -- e.g. KST-RFQ-2026-014
  title       varchar(200) NOT NULL,
  ttype       varchar(8)  NOT NULL,             -- RFI | RFP | RFQ | AUC
  category    varchar(60) NOT NULL,
  status      varchar(16) NOT NULL,             -- draft | approval | published | paused | evaluation |
  awarded | cancelled
  owner_id    varchar(16) REFERENCES core_persona(id) ON DELETE SET NULL,
  -- spend coding (shared with core_contract; values come from OrgSetting)
  department  varchar(80) NOT NULL DEFAULT '',
  cost_centre varchar(40) NOT NULL DEFAULT '',
  project     varchar(120) NOT NULL DEFAULT '',
  region      varchar(60) NOT NULL DEFAULT '',
  funding_source varchar(80) NOT NULL DEFAULT '',
  -- the three money columns
  budget      bigint NOT NULL,                  -- the ceiling somebody set
  projected_cost bigint,                       -- what the category manager expects
  baseline    bigint,                          -- what we were paying before
  baseline_source varchar(200) NOT NULL DEFAULT '',
  -- what is being bought and how it is judged
  scope       text NOT NULL DEFAULT '',
  criteria    jsonb NOT NULL DEFAULT '[]',      -- [{id, label, weight, kind}]
  lines       jsonb NOT NULL DEFAULT '[]',      -- [{id, description, qty, uom, itemCode}]
  tech_weight integer NOT NULL DEFAULT 60,
  comm_weight integer NOT NULL DEFAULT 40,
  two_stage   boolean NOT NULL DEFAULT false,
  tech_threshold integer NOT NULL DEFAULT 0,    -- min technical score to have prices opened
  auction_min_decrement bigint NOT NULL DEFAULT 0,
  -- who was asked, what they were told
  invited     jsonb NOT NULL DEFAULT '[]',      -- [supplier_id]
  addenda     jsonb NOT NULL DEFAULT '[]',
  coi         jsonb NOT NULL DEFAULT '{}',      -- {persona_id: signed_at}
  -- lifecycle: status says where; these say when, and why
  published_at bigint,
  deadline    bigint NOT NULL,
  deadline_changes jsonb NOT NULL DEFAULT '[]', -- [{from, to, at, by, reason}]
  paused_at   bigint,   paused_reason varchar(300) NOT NULL DEFAULT '',
  resumed_at  bigint,
  cancelled_at bigint,   cancel_reason  varchar(300) NOT NULL DEFAULT '',
  tech_opened_at bigint, -- stage-1 opening in a two-stage event
  opened_at   bigint,   -- the recorded opening
  -- the outcome
  award_rec   jsonb,      -- {supplier_id, amount, basis, rationale, by, at}
  awarded_at  bigint,
  awarded_to  varchar(16),
  awarded_amount bigint,
  award_memo  text NOT NULL DEFAULT '',
  letters     jsonb      -- {supplier_id: {kind: award|regret, body, at}}
);
```

WHY IT IS SHAPED THIS WAY

Three nullable money columns instead of one. Budget is a ceiling, projection is a forecast, baseline is history. If projection defaulted to the budget it would read as a forecast and not be one, so it is `NULL` until somebody enters it. The savings query picks the strongest basis present and names it.

Timestamps, not more status words. `status` says where the event is; `paused_at`, `resumed_at`, `cancelled_at` say when it got there and that it was ever paused, which a resumed event must remember for the audit trail to make sense.

Deadline changes are a list on the tender as well as events on the audit chain. "How many times has this been extended and who keeps extending it" is a question about the tender, not something to reassemble from four thousand chain rows.

Evaluation and award live here, not in their own tables. COI signatures, the recommendation and the letters are each written once per tender and read as a unit. They are JSON columns on the row they describe, and every write of them is also an audit event.

Spend coding is text, not foreign keys. The allowed values are organisation settings that change when a department is renamed, which happens more often than a deployment. A blank is reported as its own "uncoded" line rather than hidden under "Other".

core_procurementround

```
CREATE TABLE core_procurementround (
  id          varchar(16) PRIMARY KEY,
  tender_id   varchar(16) NOT NULL REFERENCES core_tender(id) ON DELETE CASCADE,
  number      integer     NOT NULL,
  name        varchar(120) NOT NULL DEFAULT '',      -- "Best and final"
  status      varchar(16) NOT NULL,                  -- draft | upcoming | open | closed | evaluation |
  completed   | cancelled
  opens_at    bigint, deadline bigint, opened_at bigint, closed_at bigint,
  instructions text NOT NULL DEFAULT '',
  invited     jsonb NOT NULL DEFAULT '[]',           -- '[]' = everyone invited to the event
  cancel_reason varchar(300) NOT NULL DEFAULT '',
  created_at   bigint NOT NULL,
  created_by   varchar(120) NOT NULL DEFAULT '',
  CONSTRAINT one_round_per_number UNIQUE (tender_id, number)
);
```

WHY IT IS SHAPED THIS WAY

Round 1 has no row until round 2 exists. An event with no rounds is an ordinary single-round competition whose window is the tender's own deadline. When somebody opens a second round, round 1 is materialised and the bids already taken are adopted into it. Nothing created before rounds existed had to be migrated.

A round is not a second tender. Making a best-and-final a new tender would fork the audit trail, the invitation list and the award, and leave nothing able to answer "what did this vendor bid last time".

core_clarification

```
CREATE TABLE core_clarification (
  id          varchar(16) PRIMARY KEY,
  tender_id   varchar(16) NOT NULL REFERENCES core_tender(id) ON DELETE CASCADE,
  supplier_id varchar(16) NOT NULL,          -- who asked; never shown to other bidders
  q           text NOT NULL,
  asked_at    bigint NOT NULL,
  a           text,                          -- NULL until answered
  answered_at bigint
);
```

WHY

The asker is stored so the buyer can reply, and stripped at serialisation so that other bidders read the answer anonymised. Blindness is enforced when the row leaves the database, not by leaving the column out.

5. Bids, auction bids and documents

core_bid

```
CREATE TABLE core_bid (
  id          varchar(16) PRIMARY KEY,
  tender_id   varchar(16) NOT NULL REFERENCES core_tender(id) ON DELETE CASCADE,
  round_id    varchar(16) REFERENCES core_procurementround(id) ON DELETE CASCADE, -- NULL = implicit
  round_id    round_id
  supplier_id varchar(16) NOT NULL,          -- text on purpose: see convention 7
  submitted_at bigint NOT NULL,
  -- sealed until the recorded opening
  sealed_blob bytea,                          -- Fernet({amount, lines}); NULL after opening
  amount      bigint,                          -- NULL while sealed
  lines       jsonb NOT NULL DEFAULT '{}', -- {line_id: unit_price}; '{}' while sealed
  -- evaluation
  scores      jsonb NOT NULL DEFAULT '{}', -- {persona_id: {criterion_id: 0..10}}
  notes       jsonb NOT NULL DEFAULT '{}', -- {persona_id: justification}
  disqualified boolean NOT NULL DEFAULT false, -- failed stage 1: commercial envelope never decrypted
  CONSTRAINT one_bid_per_supplier_round UNIQUE (tender_id, supplier_id, round_id)
);
CREATE UNIQUE INDEX one_bid_per_supplier ON core_bid (tender_id, supplier_id) WHERE round_id IS NULL;
```

WHY IT IS SHAPED THIS WAY

The price is ciphertext until opening. On submission the amount and line prices are encrypted into `sealed_blob` and the plain columns stay `NULL` and empty. A database dump taken before the opening contains no prices. At the recorded opening the blob is decrypted into `amount` and `lines`, and that act is an audit event.

A disqualified bidder's blob is never decrypted. In a two-stage event, a bid under the technical threshold keeps its `sealed_blob` and `amount` stays `NULL` forever. Exports and the API read the same columns, so they cannot show what the database does not hold in clear.

Two uniqueness rules, because SQL treats NULLs as distinct. The composite constraint would let a single-round event accept the same vendor twice with `round_id NULL`. The partial index closes that gap.

Scores are keyed by evaluator inside the row. Each evaluator is only ever sent their own key from this JSON, so blind scoring is a filter on output, and the consensus matrix is one read of one row.

core_auctionbid

```
CREATE TABLE core_auctionbid (
  id          varchar(16) PRIMARY KEY,
  tender_id   varchar(16) NOT NULL REFERENCES core_tender(id) ON DELETE CASCADE,
  supplier_id varchar(16) NOT NULL,
  amount      bigint NOT NULL,
  at          bigint NOT NULL
);
```

WHY

One row per price ever submitted, never updated. A reverse auction is a movement, and the movement is the evidence: it is what shows sniping, decrement enforcement and who held the lead when. Suppliers are sent only their rank computed from this table, never another row.

core_document

```
CREATE TABLE core_document (
  id          varchar(16) PRIMARY KEY,
  kind        varchar(12) NOT NULL,           -- tender | bid | supplier
  tender_id   varchar(16) REFERENCES core_tender(id) ON DELETE CASCADE,
  round_id    varchar(16) REFERENCES core_procurementround(id) ON DELETE SET NULL,
  supplier_id varchar(16),
  envelope    varchar(12) NOT NULL DEFAULT '', -- technical | commercial (bid docs)
  encrypted   boolean NOT NULL DEFAULT false,
  expiry      bigint,                          -- compliance documents only
  name        varchar(200) NOT NULL,
  content_type varchar(120) NOT NULL,
  size        integer NOT NULL,
  data        bytea NOT NULL,                  -- the file itself, max 10 MB
  uploaded_by varchar(120) NOT NULL,
  uploaded_at bigint NOT NULL
);
```

WHY

Files live in the database so they survive red deploys with no object storage to configure, and so a bid document is sealed by the same mechanism as a bid price: the bytes are ciphertext and `encrypted` is true until the opening. Before that, buyers are sent only the fact that a document exists, not its name.

6. The audit trail

core_event and core_chainhead

```
CREATE TABLE core_event (
  id      varchar(16) PRIMARY KEY,
  seq     integer      NOT NULL,           -- position in the chain, 1, 2, 3 ...
  prev_hash varchar(64) NOT NULL,          -- hash of seq - 1 ('genesis' for the first)
  hash     varchar(64) NOT NULL,          -- sha256 of this row plus prev_hash
  at       bigint       NOT NULL,
  actor    varchar(120) NOT NULL,          -- the name that was true at the time
  role     varchar(20)  NOT NULL,
  action   varchar(120) NOT NULL,          -- 'tender.published', 'bid.sealed', 'coi.signed' ...
  tender_id varchar(16),                  -- text: survives the tender
  detail   text         NOT NULL DEFAULT ''
);

CREATE TABLE core_chainhead (
  id integer PRIMARY KEY,                -- always 1
  seq integer NOT NULL,
  hash varchar(64) NOT NULL
);

-- how every row is written (util.record_event), inside one transaction:
-- SELECT ... FROM core_chainhead WHERE id = 1 FOR UPDATE;
-- hash = sha256(prev_hash | id | seq | at | actor | action | tender_id | detail)
-- INSERT INTO core_event ...; UPDATE core_chainhead SET seq = seq + 1, hash = ...;
```

WHY IT IS SHAPED THIS WAY

Each row carries the hash of the one before it. Change any historical value, even a single character of `detail`, and every hash after it stops matching. The auditor's one-click check recomputes the chain from the first row and names the first sequence number that breaks.

The single head row is locked while an event is written, so two users acting in the same millisecond still get consecutive sequence numbers and a single unbroken chain.

Actor is a name, tender is a text id. Renaming a person or deleting a tender must not alter or orphan a row that was true when written. Nothing here is a foreign key, and the table has no update path in the application.

core_adminaudit

```
CREATE TABLE core_adminaudit (
  id      varchar(16) PRIMARY KEY,
  at      bigint      NOT NULL,
  actor   varchar(200) NOT NULL,      -- administrator's username
  action  varchar(140) NOT NULL,
  target  varchar(200) NOT NULL DEFAULT '',
  detail  text        NOT NULL DEFAULT '',
  ip      varchar(64)  NOT NULL DEFAULT ''
);
```

WHY

Acts *on* the workspace (roles, permissions, settings) are kept apart from acts *within* it, because they are read in a different place by different people. A permission change is additionally written to the main chain, since a capability that moved during a live tender is exactly what an auditor is there to find.

7. The finance mirror

Five ledger tables share three column sets. They are shown once here and referenced below, which is also how the code defines them.

```
-- Mirrored: provenance and staleness, on every ledger row
source      varchar(12) NOT NULL,      -- nav | bc | seed | manual
external_id varchar(80) NOT NULL,      -- the finance system's own key
synced_at   bigint      NOT NULL,
UNIQUE (source, external_id)          -- re-import updates, never duplicates

-- Money: one currency in a total
amount      bigint NOT NULL DEFAULT 0,      -- NGN, the only column anything sums
currency    varchar(3) NOT NULL DEFAULT 'NGN',
amount_src  bigint NOT NULL DEFAULT 0,      -- as agreed, in 'currency'
fx_rate     double precision NOT NULL DEFAULT 1 -- NGN per unit, at commitment

-- SpendDimensions: the same five columns as core_tender
department, cost_centre, project, region, funding_source
```

WHY IT IS SHAPED THIS WAY

Provenance travels with the row. The unique key on source plus external id means the NAV feed can run nightly and update the same rows. The day the ledger moves to Business Central, its rows arrive as `bc` and the old ones stay attributable to NAV.

Staleness is a fact, not an absence. Every row knows when it was last synced, and the banner on every finance view reports the age of the stalest feed from `core_sourcesync`.

Every amount is stored twice. Summing mixed currencies is the easiest way to publish a figure wrong by an order of magnitude, so the converted naira figure is the only one that adds up. The original pair is what the exchange-rate exposure is computed from: what was struck, against the rate today.

core_contract

```
CREATE TABLE core_contract (
  id          varchar(24) PRIMARY KEY,
  -- Mirrored + Money + SpendDimensions columns, as above
  tender_id   varchar(16) REFERENCES core_tender(id) ON DELETE SET NULL, -- NULL: contract nobody
  tendered here
  supplier_id varchar(16) REFERENCES core_supplier(id) ON DELETE SET NULL,
  ref         varchar(60) NOT NULL,
  title       varchar(200) NOT NULL DEFAULT '',
  original_value bigint NOT NULL DEFAULT 0, -- value at signature; 'amount' is the value now
  signed_at   bigint, starts_at bigint, ends_at bigint,
  status      varchar(16) NOT NULL, -- active | expired | terminated | draft
  change_orders jsonb NOT NULL DEFAULT '[]', -- [{ref, at, delta, reason}]
  renewal_notice_days integer NOT NULL DEFAULT 90,
  CONSTRAINT one_contract_per_external_id UNIQUE (source, external_id)
);
```

WHY

Escalation is a subtraction, current value minus value at signature, and the individual variations are listed so "grew ₦35m" and "grew ₦35m across nine unbatched variations" are distinguishable. The nullable tender link is what lets the ledger know about spend that never went through a competition, and report it.

core_purchaseorder, core_goodsreceipt, core_invoice, core_payment

```

CREATE TABLE core_purchaseorder (
  id          varchar(24) PRIMARY KEY,    -- Mirrored + Money
  contract_id varchar(24) REFERENCES core_contract(id) ON DELETE SET NULL,
  tender_id   varchar(16) REFERENCES core_tender(id)   ON DELETE SET NULL,
  supplier_id varchar(16) REFERENCES core_supplier(id) ON DELETE SET NULL,
  ref         varchar(60) NOT NULL,  description varchar(200) NOT NULL DEFAULT '',
  raised_at   bigint,  raised_by varchar(120) NOT NULL DEFAULT '',
  status      varchar(16) NOT NULL,
  approved_at bigint,  approved_by varchar(120) NOT NULL DEFAULT '',
  CONSTRAINT one_po_per_external_id UNIQUE (source, external_id)
);

CREATE TABLE core_goodsreceipt (
  id          varchar(24) PRIMARY KEY,    -- Mirrored + Money
  order_id    varchar(24) REFERENCES core_purchaseorder(id) ON DELETE SET NULL,
  ref         varchar(60) NOT NULL,
  received_at bigint,  received_by varchar(120) NOT NULL DEFAULT '',
  note        varchar(200) NOT NULL DEFAULT '',
  CONSTRAINT one_grn_per_external_id UNIQUE (source, external_id)
);

CREATE TABLE core_invoice (
  id          varchar(24) PRIMARY KEY,    -- Mirrored + Money
  contract_id varchar(24) REFERENCES core_contract(id)      ON DELETE SET NULL,
  order_id    varchar(24) REFERENCES core_purchaseorder(id) ON DELETE SET NULL,
  receipt_id  varchar(24) REFERENCES core_goodsreceipt(id)  ON DELETE SET NULL,
  supplier_id varchar(16) REFERENCES core_supplier(id)      ON DELETE SET NULL,
  supplier_ref varchar(80) NOT NULL DEFAULT '',    -- the supplier's own invoice number
  invoiced_at bigint,
  received_at bigint,    -- the clock the supplier experiences starts here
  due_at      bigint,
  approved_at bigint,  approved_by varchar(120) NOT NULL DEFAULT '',
  status      varchar(16) NOT NULL,    -- received | approved | held | paid
  hold_reason varchar(200) NOT NULL DEFAULT '',
  discount_pct double precision NOT NULL DEFAULT 0,    -- early-payment terms offered
  discount_days integer NOT NULL DEFAULT 0,
  CONSTRAINT one_invoice_per_external_id UNIQUE (source, external_id)
);

CREATE TABLE core_payment (
  id          varchar(24) PRIMARY KEY,    -- Mirrored + Money
  invoice_id  varchar(24) REFERENCES core_invoice(id) ON DELETE SET NULL,
  supplier_id varchar(16) REFERENCES core_supplier(id) ON DELETE SET NULL,
  ref         varchar(60) NOT NULL,
  paid_at     bigint,
  method      varchar(24) NOT NULL DEFAULT '',
  discount_taken bigint NOT NULL DEFAULT 0,
  CONSTRAINT one_payment_per_external_id UNIQUE (source, external_id)
);

```

WHY IT IS SHAPED THIS WAY

The three-way match is three nullable keys on the invoice. An invoice that points at an order and a receipt is matched; one with a missing link is an exception the sweep raises. Making the links optional is what lets a broken match exist as a row to be found rather than a row that could not be imported.

Received date and invoice date are both kept. Payment performance is measured from receipt, because that is the wait the supplier actually experiences, and measuring from approval would make the slowest step invisible.

Discount offered and discount taken are on different tables. Offered lives on the invoice, taken on the payment. The gap between them, per invoice, is the "missed discount" figure, a real loss the mirror can name only because both sides are stored.

Duplicate invoices are catchable by the supplier's own reference and by same supplier, same amount, same day, because `supplier_ref`, `amount` and `invoiced_at` are all plain indexed columns.

core_item, core_fxrate, core_sourcesync

```
CREATE TABLE core_item (
    id          varchar(24) PRIMARY KEY, -- the material master, mirrored
    code        varchar(40) NOT NULL,    -- Mirrored
    description  varchar(200) NOT NULL,    description2 varchar(200) NOT NULL DEFAULT '',
    uom          varchar(24) NOT NULL DEFAULT '', -- each | carton | kg
    category     varchar(80) NOT NULL DEFAULT '',
    unit_cost    bigint NOT NULL DEFAULT 0,    currency varchar(3) NOT NULL DEFAULT 'NGN',
    blocked      boolean NOT NULL DEFAULT false, -- discontinued: kept for history, not offered on new
    tenders
    kind         varchar(24) NOT NULL DEFAULT '',
    CONSTRAINT one_item_per_external_id UNIQUE (source, external_id)
);
CREATE INDEX ON core_item (code);

CREATE TABLE core_fxrate (
    id          integer GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    currency    varchar(3) NOT NULL,
    at          bigint NOT NULL,           -- one observation per day
    rate        double precision NOT NULL, -- NGN per unit
    source      varchar(12) NOT NULL,
    CONSTRAINT one_rate_per_day UNIQUE (currency, at)
);

CREATE TABLE core_sourcesync (
    id          integer GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    source       varchar(12) NOT NULL,     -- nav | bc
    entity       varchar(24) NOT NULL,     -- contract | po | grin | invoice | payment | item | fx
    last_attempt bigint NOT NULL,
    last_success bigint NOT NULL,
    rows_seen    integer NOT NULL,    rows_written integer NOT NULL,
    error        varchar(300) NOT NULL DEFAULT '',
    CONSTRAINT one_sync_per_feed UNIQUE (source, entity)
);
```

WHY IT IS SHAPED THIS WAY

Items exist so that tender lines have a "this". Before the item master was mirrored, a line was free text and no two tenders described the same oven the same way, which made "are we paying more than last year" unanswerable. A tender line that carries an item code can be compared across years.

Exchange rates are history, not a current value, because exposure is only actionable as a movement: struck at 1,480, today 1,655.

Last attempt and last success are separate columns. A feed that silently stopped three weeks ago looks exactly like a quiet month unless the failed attempt is recorded next to the last success.

8. People, roles and access

```
CREATE TABLE core_person ( -- a buyer-side person
  id          varchar(16) PRIMARY KEY,
  name        varchar(80) NOT NULL,
  role        varchar(20) NOT NULL, -- procurement | evaluator | approver | auditor | a custom key
  title       varchar(80) NOT NULL,
  manager_id  varchar(16) REFERENCES core_person(id) ON DELETE SET NULL
);

CREATE TABLE core_profile ( -- joins a login to an identity
  id          integer GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  user_id     integer NOT NULL UNIQUE REFERENCES auth_user(id) ON DELETE CASCADE,
  persona_id  varchar(16) REFERENCES core_person(id) ON DELETE SET NULL,
  supplier_id varchar(16) REFERENCES core_supplier(id) ON DELETE SET NULL,
  totp_secret varchar(64) NOT NULL DEFAULT '',
  totp_confirmed boolean NOT NULL DEFAULT false,
  perm_extra  jsonb NOT NULL DEFAULT '[]', -- capabilities granted beyond the role
  perm_revoked jsonb NOT NULL DEFAULT '[]' -- capabilities taken from the role
);

CREATE TABLE core_accessrole ( -- roles invented in the console: "Legal", "Board"
  key          varchar(20) PRIMARY KEY, -- lands in core_person.role
  label        varchar(80) NOT NULL, title varchar(80) NOT NULL DEFAULT '',
  note         varchar(200) NOT NULL DEFAULT '',
  perms        jsonb NOT NULL DEFAULT '[]', -- capability keys, e.g. 'tender.extend', 'finance.payables'
  created      bigint NOT NULL, created_by varchar(200) NOT NULL DEFAULT ''
);

CREATE TABLE core_authtoken ( -- opaque bearer token, one per sign-in
  key          varchar(64) PRIMARY KEY,
  user_id     integer NOT NULL REFERENCES auth_user(id) ON DELETE CASCADE,
  created      bigint NOT NULL, last_used bigint NOT NULL
);

CREATE TABLE core_actiontoken ( -- single-use emailed links
  token        varchar(64) PRIMARY KEY,
  kind         varchar(20) NOT NULL, -- vendor_verify | vendor_invite | team_invite | reset
  email        varchar(200) NOT NULL,
  payload      jsonb NOT NULL DEFAULT '{}',
  created      bigint NOT NULL, used_at bigint -- NULL until spent
);
```

WHY IT IS SHAPED THIS WAY

A profile with no persona and no supplier is an administrator: an account that manages people and permissions and takes no part in tendering. The two nullable keys are the whole role model for who a login is.

Role and reporting line are separate columns. A role says what you may do; a manager says whose work you may see. The CFO and a buyer can hold the same capability and still owe an account to different people.

Permissions are role defaults plus two lists of exceptions. The four built-in roles are code, because separation of duties is the product. Custom roles are configuration. Moving one person off their role's defaults is two JSON lists on their profile, and every such change is an admin-audit row.

Tokens are rows, so they can be revoked. Sign-out deletes the row. "Sign out everywhere" deletes every row for the user. A password reset deletes them all. A stateless token could do none of that.

9. System tables

```
CREATE TABLE core_notification (
  id          varchar(16) PRIMARY KEY,
  user_id     integer NOT NULL REFERENCES auth_user(id) ON DELETE CASCADE,
  at          bigint NOT NULL,
  subject     varchar(200) NOT NULL, body text NOT NULL,
  tender_id   varchar(16),
  read        boolean NOT NULL DEFAULT false,
  emailed     boolean NOT NULL DEFAULT false      -- proof the email step ran
);

CREATE TABLE core_failedlogin ( id integer GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
  username varchar(200) NOT NULL, at bigint NOT NULL ); -- 5 in 15 min locks the name
CREATE INDEX ON core_failedlogin (username);

CREATE TABLE core_taskmark ( key varchar(120) PRIMARY KEY, at bigint NOT NULL ); -- sweep idempotence
CREATE TABLE core_orgsetting ( id integer PRIMARY KEY, data jsonb NOT NULL DEFAULT '{}'); --
{approvalThreshold, dimensions, workspaceName}
CREATE TABLE core_demofixture( id integer PRIMARY KEY, at bigint NOT NULL, manifest jsonb NOT NULL
DEFAULT '{}');
```

WHY

Task marks are why a notification fires once per finding, not once per sweep. Each rule writes a key such as `ex:duplicate_invoice:INV-0093` when it notifies; the next sweep sees the key and stays quiet.

The demo fixture is a manifest, not a pattern. The seed writes down exactly which ids it created so it can be removed exactly. Deleting "everything that looks like demo data" works until the day a real vendor is called Coldline Logistics.

10. The questions, as queries

The first document listed uses as questions. Here are five of them written against the tables above, to show the shapes carry the weight. Amounts are naira; timestamps are divided by 1,000 to reach seconds.

Savings this year, with the basis named and never summed

```
SELECT
  ref, title, awarded_amount,
  CASE WHEN baseline IS NOT NULL THEN 'negotiated' ELSE 'against budget' END AS basis,
  COALESCE(baseline, budget) - awarded_amount AS saving
FROM core_tender
WHERE status = 'awarded'
  AND to_timestamp(awarded_at / 1000) >= date_trunc('year', now())
ORDER BY awarded_at;
```

Awarded spend by department, with uncoded spend as its own line

```
SELECT
  COALESCE(NULLIF(department, ''), '(uncoded)') AS department,
  COUNT(*) AS tenders,
  SUM(awarded_amount) AS awarded_ngn
FROM core_tender
WHERE status = 'awarded'
GROUP BY 1
ORDER BY awarded_ngn DESC;
```

Days to pay, measured from receipt of the invoice

```
SELECT
  s.name AS supplier,
  COUNT(p.id) AS invoices_paid,
  ROUND(AVG((p.paid_at - i.received_at) / 86400000.0), 1) AS mean_days,
  PERCENTILE_CONT(0.5) WITHIN GROUP (ORDER BY (p.paid_at - i.received_at) / 86400000.0) AS median_days,
  SUM(CASE WHEN i.discount_pct > 0 AND p.discount_taken = 0 THEN i.amount * i.discount_pct / 100 ELSE 0
END) AS discount_missed_ngn
FROM core_invoice i
JOIN core_payment p ON p.invoice_id = i.id
JOIN core_supplier s ON s.id = i.supplier_id
WHERE i.received_at IS NOT NULL AND p.paid_at IS NOT NULL
GROUP BY s.name
ORDER BY median_days DESC;
```

Single-bidder awards in the last twelve months

```
SELECT t.ref, t.title, t.awarded_amount, COUNT(b.id) AS bids
FROM core_tender t
LEFT JOIN core_bid b ON b.tender_id = t.id AND b.round_id IS NULL
WHERE t.status = 'awarded'
  AND t.awarded_at > (EXTRACT(EPOCH FROM now()) * 1000 - 365 * 86400000)::bigint
GROUP BY t.id
HAVING COUNT(b.id) = 1;
```

Is the audit chain intact? The first row whose link is broken

```
WITH ordered AS (
  SELECT seq, prev_hash, hash,
         LAG(hash) OVER (ORDER BY seq) AS expected_prev
  FROM core_event
)
SELECT seq FROM ordered
WHERE prev_hash <> COALESCE(expected_prev, 'genesis')
ORDER BY seq LIMIT 1;
-- empty result = every link matches. The application also recomputes each
-- row's own hash from its contents, which SQL alone cannot, so the in-app
-- verification is the authoritative one; this query is the quick check.
```

Which is the point of the exercise. None of these queries needs a table that does not exist, a column that would have to be parsed out of free text, or a join across a currency boundary. The conventions in section 1 are what make each of them a few lines.

11. What is deliberately not a column

NOT STORED	WHERE YOU MIGHT EXPECT IT	WHY IT IS ABSENT
bank_account	core_supplier	DOCKET awards and does not pay. Only the last four digits are kept, inside the registry JSON.
amount (before opening)	core_bid	Held only as ciphertext in <code>sealed_blob</code> . The plain column is <code>NULL</code> until the recorded opening, so no query, export or dump can reveal it early.
amount (disqualified bidder)	core_bid	Never decrypted. The commercial envelope of a bidder who failed stage 1 stays sealed permanently.
total_savings	core_tender or a summary table	The three savings figures are computed at read time and never added. A stored total would be a headline nobody could defend.
risk_score, fraud_score	core_supplier	DOCKET cannot see a balance sheet. It stores observations (expired documents, concentration, abnormal bids) and lets a person draw the conclusion.
FOREIGN KEY on bid.supplier_id	core_bid	So a register re-import can never cascade-delete bid history. The import refuses to remove a referenced vendor instead.
updated_at on core_event	core_event	Audit rows have no update path. The hash chain makes any edit detectable, so there is nothing legitimate to timestamp.
competitor_rank / competitor_price	core_auctionbid	Rank is computed per request from the bids table and sent only to the supplier it belongs to. Nothing about another bidder is ever materialised onto a supplier's row.

Column lists were generated from the DOCKET Django models with the schema editor on 14 September 2026 and rendered here in PostgreSQL syntax. Inline comments and the reasons given are drawn from the comments in the model source, the README, and the finance and audit modules. Where a default is shown, it is the model default; Django applies most of these at insert time rather than as database defaults.